```
        } catch (GeneralSecurityException e) {
            throw new RuntimeException(e);
        }
    }

    @Override
    public Optional<Token> read(Request request, String
tokenId) {
        return Optional.empty(); // To be written
    }
}
```

#A Pass in the real TokenStore implementation and the secret key to the constructor.
#B Call the real TokenStore to generate the token ID, then use HMAC to calculate the tag.
#C Concatenate the original token ID with the encoded tag as the new token ID.
#D Use the javax.crypto.Mac class to compute the HMAC-SHA256 tag.

When the client presents the token back to the API, you extract the tag from the presented token and recompute the expected tag from the secret and the rest of the token ID. If they match then the token is authentic, and you pass it through to the `DatabaseTokenStore`. If they don’t match, then the request is rejected. Listing 5.11 shows the code to validate the tag. First you need to extract the tag from the token and decode it. You then compute the correct tag just as you did when creating a fresh token and check the two are equal.